OnDeviceTraining

Code Reading Guide 2

Line-by-Line Syntax & Explanation

Mohamed · Master's Thesis · 2026

This guide walks through every key source file in the codebase, presenting the actual C code with a per-line syntax label and a plain-English explanation of what each construct does and why it exists. Read Guide 1 first to understand the phase structure; use this guide to understand the code itself.

1 · Common.h

`src/common/include/Common.h`

Common.h is a header-only file included by every other .c file in the project. It provides three debug-print macros and one raw byte-dump macro, all controlled by a compile-time verbosity level. Nothing here is a function — every construct is a preprocessor macro.

`#ifndef ODT_COMMON_H`	■ Include guard — prevents double inclusion. If the symbol ODT_COMMON_H is not yet defined, process the file.
`#define ODT_COMMON_H`	■ Define the guard symbol so any subsequent #include of this header is a no-op.
`#include <stdbool.h>`	■ Pulls in bool, true, false — used by the do{}while(false) idiom below.
`#include <stdio.h>`	■ Pulls in printf() — the underlying output function for all macros.
`#include <string.h>`	■ Pulls in string utilities (not used directly here, but transitively needed by callers).
`#ifndef SOURCE_FILE`	■ If the .c file forgot to #define SOURCE_FILE before including Common.h, provide a fallback string.
`#define SOURCE_FILE "no Source file defined!"`	■ Fallback string literal for SOURCE_FILE. Each .c sets its own, e.g. #define SOURCE_FILE "ROUNDING".
`#endif`	■ Closes the #ifndef SOURCE_FILE guard.
`#ifdef DEBUG_MODE_DEBUG`	■ If -DDEBUG_MODE_DEBUG was passed to the compiler, set DLEVEL=3 (most verbose).
`#define DLEVEL 3`	■ DLEVEL 3 enables DEBUG + INFO + ERROR messages.
`#elif defined(DEBUG_MODE_INFO)`	■ Otherwise, check for -DDEBUG_MODE_INFO.
`#define DLEVEL 2`	■ DLEVEL 2 enables INFO + ERROR messages only.
`#elif defined(DEBUG_MODE_ERROR)`	■ Otherwise, check for -DDEBUG_MODE_ERROR.
`#define DLEVEL 1`	■ DLEVEL 1 enables ERROR messages only.
`#else`	■ No debug flag set — silent mode.
`#define DLEVEL 0`	■ DLEVEL 0 suppresses all output (default for release / MCU builds).
`#endif`	■ Closes the debug-level ladder.

The three debug macros all use the same pattern — a do{}while(false) block that guards against misuse:

`#define PRINT_DEBUG(str, ...) \`	■ Macro definition with variadic args (...). The backslash continues the definition onto the next line.
`do { \`	■ do{...}while(false) wraps a multi-statement block so it behaves like a single statement (safe after an if without braces).
`if (DLEVEL >= 3) { \`	■ Runtime check — only print if the active verbosity level is 3 or higher.
`printf("\033[0;33m[%s: %s] ", SOURCE_FILE, __FUNCTION__); \`	■ Print a yellow ANSI prefix (\033[0;33m), the source file name, and the C function name (__FUNCTION__ is a predefined identifier).

`printf(str, ##__VA_ARGS__); \`	■ Print the user's format string with any additional arguments. ##__VA_ARGS__ swallows the leading comma if no args are given.
`printf("\033[0m\n"); \`	■ Reset the ANSI colour (\033[0m) and print a newline.
`} \`	■ Close the if block.
`} while (false)`	■ The while(false) ensures the macro expands to a single expression — critical for correct if/else parsing.

✔ PRINT_INFO is identical but uses DLEVEL >= 2 and no colour codes. PRINT_ERROR uses DLEVEL >= 1 and red (\033[0;31m).

`#define PRINT_BYTE_ARRAY(prefix, byteArray, numberOfBytes) \`	■ Macro to dump raw bytes — no DLEVEL guard, always prints.
`do { \`	■ Same do{}while pattern.
`printf("[%s: %s] ", SOURCE_FILE, __FUNCTION__); \`	■ Print file/function prefix.
`printf(prefix); \`	■ Print caller-supplied label string.
`for (size_t index = 0; index < numberOfBytes; index++) { \`	■ Loop over each byte.
`printf("0x%02X ", byteArray[index]); \`	■ Print each byte as a zero-padded two-digit hex value (e.g. 0x3F).
`} \`	
`printf("\n"); \`	■ Newline after the dump.
`} while (false)`	

2 · Rounding.h / Rounding.c

`src/arithmetic/include/Rounding.h + src/arithmetic/Rounding.c`

Rounding provides the two rounding strategies used throughout quantization: Half-To-Even (HTE, standard banker's rounding) and Stochastic Rounding Half-To-Even (SRHTE). SRHTE is the core trick of Fully Quantized Training — it adds a random dither before rounding so that small gradients are not always floored to zero.

Header — Rounding.h:

`#include <stdint.h>`	■ Provides int32_t — the return type of roundByMode.
`typedef enum roundingMode {`	■ Declare a named enum type. typedef lets us use roundingMode_t directly instead of enum roundingMode.
`HTE,`	■ Enumerator 0. Half-To-Even: round 0.5 to the nearest even integer. Deterministic.
`SRHTE`	■ Enumerator 1. Stochastic Rounding: adds a uniform random value in (-0.5, 0.5) before rounding. Non-deterministic.
`} roundingMode_t;`	■ Closes the enum and creates the alias roundingMode_t.
`int32_t roundByMode(float input, roundingMode_t roundingMode);`	■ Function declaration. Takes a float and a mode; returns the rounded integer. Defined in Rounding.c.
`float clamp(float input, float min, float max);`	■ Declaration of the saturate helper — clamps a float to [min, max].

Implementation — Rounding.c:

`#define SOURCE_FILE "ROUNDING"`	■ Sets the source file label used by PRINT_DEBUG/ERROR macros in Common.h.
`#include <math.h>`	■ Needed for round() — the C standard library rounding function.

`#include "Rounding.h"`	■ Include own header — gives access to roundingMode_t declaration.
`#include "RNG.h"`	■ Include the RNG module — SRHTE needs rngNextFloat() for its random dither.
`int32_t roundHTE(float input) {`	■ Internal (no header declaration) HTE implementation.
`return round(input);`	■ Calls C99 round(), which ties away from zero — close enough to banker's rounding for all non-0.5 cases.
`}`	
`float randfloat() {`	■ Thin wrapper over rngNextFloat() so Rounding.c does not expose RNG internals.
`return rngNextFloat();`	■ Returns a float in [0, 1) from the module-global XOR-shift RNG.
`}`	
`int32_t roundSRHTE(const float input) {`	■ Stochastic rounding: shift input by (random - 0.5) before rounding.
`return roundHTE(input + randfloat() - 0.5f);`	■ randfloat() is in [0,1), so randfloat()-0.5f is in [-0.5, 0.5). Adding this dither means a value of 0.1 will round to 1 about 10% of the time and to 0 about 90% of the time — the expected value is preserved.
`}`	
`int32_t roundByMode(const float input, const roundingMode_t roundingMode) {`	■ Public dispatch function. const prevents accidental modification.
`switch (roundingMode) {`	■ Switch on the enum value — compiler can optimise to a jump table.
`case HTE:`	
`return roundHTE(input);`	■ Deterministic path.
`case SRHTE:`	
`return roundSRHTE(input);`	■ Stochastic path — used during FQT backward passes.
`}`	
`return 0;`	■ Unreachable, but suppresses compiler warnings about missing return.
`}`	
`float clamp(float input, float min, float max) {`	■ Saturate function.
`if (input < min) { return min; }`	■ If below range, return the lower bound.
`if (input > max) { return max; }`	■ If above range, return the upper bound.
`return input;`	■ Otherwise pass through unchanged.
`}`	

✔ *Why SRHTE matters: In standard int8 training, gradients smaller than 0.5 LSB always round to zero and vanish. SRHTE preserves their expected value, allowing the network to learn from tiny signals — essential for FQT.*

---

3 · DTypes.h / DTypes.c

`src/tensor/include/DTypes.h + src/tensor/DTypes.c`

DTypes provides portable read/write helpers for int32 and float32 values stored as raw byte arrays (uint8_t*). The key technique is memcpy-based type punning — the only portable way in C to reinterpret the bit pattern of a float as an int or vice versa.

`int32_t readBytesAsInt32(uint8_t *bytes) {`	■ Read 4 consecutive bytes and interpret them as a 32-bit signed integer.

`int32_t x;`	■ Declare a local int32 to receive the value.
`memcpy(&x, bytes, sizeof(int32_t));`	■ memcpy copies 4 bytes from the byte array into x's memory without triggering undefined behaviour from aliasing. sizeof(int32_t)==4 always.
`return x;`	■ Return the reconstructed integer.
`}`	
`float readBytesAsFloat(uint8_t *bytes) {`	■ Same pattern for float32 — copies 4 bytes into a float variable.
`float x;`	
`memcpy(&x, bytes, sizeof(float));`	■ memcpy is the only C-standard-compliant way to reinterpret bytes as a float.
`return x;`	
`}`	
`void readBytesAsInt32Array(size_t numberOfValues, uint8_t *bytes, int32_t *outputArray) {`	■ Batch version — reads n int32 values from a packed byte buffer.
`for (size_t i = 0; i < numberOfValues; i++) {`	■ Iterate over each element.
`size_t byteIndex = i * sizeof(int32_t);`	■ Each int32 occupies 4 bytes, so the i-th element starts at byte 4*i.
`int32_t value = readBytesAsInt32(&bytes[byteIndex]);`	■ Delegate to the single-value helper.
`outputArray[i] = value;`	■ Store in caller-supplied output array.
`}`	
`}`	
`void writeInt32ToByteArray(int32_t value, uint8_t *bytes) {`	■ Write one int32 as 4 bytes into a byte buffer.
`memcpy(bytes, &value, sizeof(int32_t));`	■ Take the address of value and copy its 4 bytes into the buffer. Endianness matches the host — consistent because read and write use the same platform.
`}`	

✔ *The memcpy idiom avoids C's strict aliasing rule: casting a float* to int* and dereferencing is undefined behaviour; memcpy is not.*

4 · DataStorage.h

`src/tensor/include/DataStorage.h (struct definitions only)`

DataStorage defines the two structs that back the static arena allocator. The file currently has no .c implementation — it is a pure header.

`typedef struct Entry {`	■ Begin a struct definition. typedef means we can write dataEntry_t instead of struct Entry.
`uint8_t* dataPTR;`	■ Pointer to the start of one allocation inside the arena's flat byte buffer.
`size_t numberOfElements;`	■ How many logical elements this entry holds (not bytes — the caller knows the element size).
`} dataEntry_t;`	■ Close struct and create alias dataEntry_t.
`typedef struct DataStorage {`	■ The arena itself.
`uint8_t *data;`	■ Pointer to the start of the backing flat byte array (allocated once at startup).
`size_t size;`	■ Total capacity of the arena in bytes.
`dataEntry_t *entries;`	■ Array of entry records — one record per allocation made from this arena.
`size_t numberOfEntries;`	■ Current number of live allocations.
`} dataStorage_t;`	■ Alias for the arena struct.
`uint8_t* getDataFromStorage(dataStorage_t storage, void* dataPTR);`	■ Lookup: given a pointer, find the corresponding arena entry.
`dataEntry_t* addDataToStorage(dataStorage_t storage, void* dataPTR, size_t numberOfElements);`	■ Register a new allocation in the entry table.

✔ *The arena pattern eliminates heap fragmentation — critical on the STM32 Nucleo-F746ZG which has 320 KB SRAM and no virtual memory.*

---

5 · StorageApi.h / StorageApi.c

`src/userApi/include/StorageApi.h + src/userApi/StorageApi.c`

StorageApi is the single entry point for all dynamic allocations in the project. On a microcontroller it maps to a static arena bump-allocator; on a host PC it wraps malloc/free. Every layer constructor, tensor initialiser, and optimizer setup function calls reserveMemory() instead of malloc() directly.

`void* reserveMemory(size_t size);`	■ Allocate size bytes. Returns a pointer — same signature as malloc so host builds can substitute malloc transparently.
`void freeReservedMemory(void* ptr);`	■ Free a previously reserved block — maps to free() on host, no-op on MCU arena where memory is never individually freed.

■ Every subsequent module allocates through reserveMemory(). Never call malloc() directly — it would bypass the arena on embedded targets.

6 · Quantization.h / Quantization.c

`src/tensor/include/Quantization.h + src/tensor/Quantization.c`

Quantization.h defines all quantization types and their config structs. The five qtypes are: INT32 (raw integer, no scaling), FLOAT32 (native float), SYM_INT32 (symmetric quantization accumulating into int32), SYM (symmetric int8), and ASYM (asymmetric int8 with zero-point). Each has an associated config struct and a set of initialiser functions.

`typedef enum qtype {`	■ Enum for the five quantization flavours.
`INT32,`	■ Value 0. Raw 32-bit integers; no scale factor. Used for intermediate accumulator tensors.
`FLOAT32,`	■ Value 1. Native 32-bit floats; no quantization error.
`SYM_INT32,`	■ Value 2. Symmetric quantization: real = int_value * scale. int32 storage. Used in FQT forward/backward.
`SYM,`	■ Value 3. Symmetric int8 (scale only, no zero-point). Not the primary path — SYM_INT32 is preferred.
`ASYM`	■ Value 4. Asymmetric int8: real = (int_value + zeroPoint) * scale. Used for activations in asymmetric inference.
`} qtype_t;`	
`typedef struct symInt32QConfig {`	■ Config for SYM_INT32 quantization.
`float scale;`	■ The scale factor: real_value = int32_value * scale.
`roundingMode_t roundingMode;`	■ HTE or SRHTE — determines how floats are quantized to int32.
`uint8_t qMaxBits;`	■ Number of significant bits in the quantized representation (default 16 for SYM_INT32).
`} symInt32QConfig_t;`	
`typedef struct asymQConfig {`	■ Config for ASYM quantization.
`float scale;`	■ Scale factor.
`int16_t zeroPoint;`	■ Zero-point offset: real = (q + zeroPoint) * scale. Allows asymmetric ranges like [0, 255].
`uint8_t qBits;`	■ Number of bits per element (typically 8).
`roundingMode_t roundingMode;`	
`} asymQConfig_t;`	
`typedef struct quantization {`	■ The universal quantization descriptor attached to every tensor.
`qtype_t type;`	■ Which of the five qtypes this is.
`void* qConfig;`	■ Pointer to the type-specific config struct (symInt32QConfig_t*, asymQConfig_t*, or NULL for INT32/FLOAT32).
`} quantization_t;`	

Key initialiser functions from Quantization.c:

`void initFloat32Quantization(quantization_t *quantization) {`	■ Initialise a FLOAT32 quantization — the simplest case.

`quantization->type = FLOAT32;`	■ Set the type discriminator.
`quantization->qConfig = NULL;`	■ No config struct needed for native floats.
`}`	
`void initSymInt32QConfig(roundingMode_t roundingMode, symInt32QConfig_t *symInt32QConfig) {`	■ Populate a SYM_INT32 config with defaults.
`symInt32QConfig->roundingMode = roundingMode;`	■ Store the caller-chosen rounding mode.
`symInt32QConfig->scale = 1.f;`	■ Initial scale=1 — will be updated at quantization time based on the tensor's actual value range.
`symInt32QConfig->qMaxBits = 16;`	■ Default to 16 significant bits (suitable for gradient accumulation).
`}`	
`void initAsymQConfig(uint8_t qBits, roundingMode_t roundingMode, asymQConfig_t *asymQConfig) {`	
`asymQConfig->qBits = qBits;`	■ Store the bit-width (typically 8).
`asymQConfig->roundingMode = roundingMode;`	
`asymQConfig->scale = 1.f;`	■ Placeholder — recomputed during convertFloatTensorToAsymTensor.
`asymQConfig->zeroPoint = (uint16_t)0;`	■ Placeholder zero-point — recomputed from the tensor's min value.
`}`	

7 · Tensor.h / Tensor.c

`src/tensor/include/Tensor.h + src/tensor/Tensor.c`

tensor_t is the central data structure. Every layer, loss function, and optimizer works exclusively with tensor_t pointers. A tensor is a flat byte buffer (data) annotated with a shape, a quantization descriptor, and an optional sparsity mask hook.

Struct definitions — Tensor.h:

`typedef struct shape {`	■ Describes the dimensionality and layout of a tensor.
`size_t numberOfDimensions;`	■ Rank of the tensor (1 for vectors, 2 for matrices, etc.).
`size_t* dimensions;`	■ Array of length numberOfDimensions — each entry is the size of that dimension.
`size_t* orderOfDimensions;`	■ Permutation array — enables transposition without copying data. orderOfDimensions[i]=j means logical dimension i maps to physical dimension j.
`} shape_t;`	
`typedef struct tensor {`	■ The main tensor struct.
`uint8_t* data;`	■ Raw backing storage. uint8_t* is used because the element size varies (4 bytes for float/int32, 1 byte for int8). DTypes helpers do the type-aware reading.
`shape_t* shape;`	■ Pointer to the shape — shared between tensors that are views of the same data.
`quantization_t* quantization;`	■ Quantization descriptor — tells consumers how to interpret data bytes as numbers.
`sparsity_t* sparsity;`	■ Future hook for RigL sparse masks. Currently NULL in all live tensors.
`} tensor_t;`	

`typedef struct parameter {`	■ Pairs a weight tensor with its gradient tensor.
`tensor_t* param;`	■ The weight values.
`tensor_t* grad;`	■ The accumulated gradient — zeroed by sgdZeroGrad() after each optimizer step.
`} parameter_t;`	

Key utility functions — Tensor.c:

`size_t calcNumberOfElementsByShape(shape_t *shape) {`	■ Compute total element count by multiplying all dimensions.
`size_t numElem = 1;`	■ Start with 1 (product identity).
`for (size_t i = 0; i < shape->numberOfDimensions; i++) {`	■ Iterate over each dimension.
`numElem *= shape->dimensions[i];`	■ Multiply — e.g. shape [784] gives 784; shape [2, 10] gives 20.
`}`	
`return numElem;`	
`}`	
`size_t calcBytesPerElement(quantization_t *quantization) {`	■ Returns bytes per element based on qtype.
`switch (quantization->type) {`	■ Dispatch on qtype.
`case INT32: return sizeof(int32_t);`	■ 4 bytes.
`case FLOAT32: return sizeof(float);`	■ 4 bytes.
`case SYM_INT32: return sizeof(int32_t);`	■ 4 bytes (int32 accumulator).
`case ASYM:`	■ Variable-width — check qBits config.
`asymQConfig_t *aq = quantization->qConfig;`	■ Cast void* to the typed config.
`return ceil((float)aq->qBits / 8.0f);`	■ Round up to whole bytes (e.g. 8 bits = 1 byte, 4 bits = 1 byte, 12 bits = 2 bytes).
`default: exit(1);`	■ Unknown type — abort.
`}`	
`}`	
`void transposeTensor(tensor_t *tensor, size_t dim0Index, size_t dim1Index) {`	■ Logical transpose — swaps two entries in orderOfDimensions without touching data.
`size_t temp = tensor->shape->orderOfDimensions[dim0Index];`	■ Save dim0's physical mapping.
`tensor->shape->orderOfDimensions[dim0Index] = tensor->shape->orderOfDimensions[dim1Index];`	■ Swap.
`tensor->shape->orderOfDimensions[dim1Index] = temp;`	■ Complete the swap.
`}`	■ This O(1) operation enables matmul to treat weights as W^T during forward and W during backward without copying.

setTensorValues and setParameterValues — lightweight struct field assignments:

`void setTensorValues(tensor_t *tensor, uint8_t *data, shape_t *shape,`	■ Fill all four fields of a tensor_t at once.
`quantization_t *quantization, sparsity_t *sparsity) {`	■ All parameters are pointers — no copying of the underlying arrays.
`tensor->data = data;`	■ Point to the backing storage.
`tensor->shape = shape;`	

`tensor->quantization = quantization;`	
`tensor->sparsity = sparsity;`	■ NULL if no sparsity mask.
`}`	

✔ Stack-allocated tensors are common in the codebase: the .c file declares 'float buf[n]; tensor_t t; setTensorValues(&t;, (uint8_t)buf, ...)' — no heap involved.*

---

8 · TensorConversion.h / TensorConversion.c

`src/tensor/include/TensorConversion.h + src/tensor/TensorConversion.c`

TensorConversion implements a 5×5 dispatch table mapping every pair of qtypes to a conversion function. The public API is a single call: convertTensor(input, output). This is the boundary layer between float and quantized arithmetic throughout the project.

`typedef void (*conversionFunction_t)(tensor_t* inputTensor, tensor_t* outputTensor);`	■ Function-pointer type alias. Any function that takes two tensor_t* and returns void can be stored here.
`void convertTensor(tensor_t* inputTensor, tensor_t* outputTensor);`	■ Public API — looks up the right converter and calls it.
`extern conversionFunction_t conversionMatrix[5][5];`	■ Declared extern so other files can inspect the table directly if needed.

The dispatch table (from TensorConversion.c):

`conversionFunction_t conversionMatrix[5][5] = {`	■ 5x5 array of function pointers. Indexed by [input_qtype][output_qtype].
`[INT32] = {`	■ Row for INT32 source tensors. C99 designated initializers allow naming the index.
`[FLOAT32] = convertInt32TensorToFloatTensor,`	■ INT32 -> FLOAT32: cast each int32 element to float.
`[SYM_INT32]= convertInt32TensorToSymInt32Tensor,`	■ INT32 -> SYM_INT32: copy data, set scale=1.
`[ASYM] = convertInt32TensorToAsymTensor,`	■ INT32 -> ASYM: compute min/max, derive scale and zeroPoint, pack to qBits.
`[SYM] = unsupportedConversionTypes,`	■ Not implemented — prints an error and exits.
`[INT32] = NULL,`	■ Same-type diagonal — handled by convertTensorsWithSameType, not the table.
`},`	
`[FLOAT32] = {`	■ Row for FLOAT32 source tensors.
`[SYM_INT32]= convertFloatTensorToSymInt32Tensor,`	■ Most important conversion in FQT: float -> symmetric int32 with per-tensor scale.
`[ASYM] = convertFloatTensorToAsymTensor,`	■ float -> asymmetric int8 with scale and zero-point.
`...`	
`},`	
`};`	

The key float→SYM_INT32 conversion:

`void convertFloatTensorToSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	
`float absMax = findAbsMaxFloat(inputTensor->data, numberOfElements);`	■ Find the largest absolute value — defines the representable range.
`const float qMax = powf(2, (float)qMaxBits - 1) - 1;`	■ Maximum positive quantized value (e.g. qMaxBits=16 -> qMax=32767).
`scale = absMax / qMax;`	■ Scale factor: maps absMax -> qMax. All values fit within [-qMax, qMax].

`outputSymInt32QC->scale = scale;`	■ Store scale in the output tensor's config so downstream code can dequantize.
`for (size_t i = 0; i < numberOfElements; i++) {`	
`outputInt32[i] = roundByMode(`	
`clamp(inputFloat[i] / scale, qMin, qMax),`	■ Divide by scale to map to integer range, then clamp to prevent overflow.
`outputSymInt32QC->roundingMode);`	■ Apply HTE or SRHTE rounding to convert the float to an integer.
`}`	
`}`	

9 · Arithmetic.h / Arithmetic.c

`src/arithmetic/include/Arithmetic.h + src/arithmetic/Arithmetic.c`

Arithmetic.c is the dispatch engine for all element-wise operations. It defines two function-pointer typedefs and generic point-wise drivers that iterate over tensor elements, handling transposition via the orderOfDimensions array. All Add, Sub, Mul, Div operations are thin wrappers over these drivers.

`typedef int32_t(*int32ElementArithmeticFunc_t)(int32_t a, int32_t b);`	■ Function-pointer type: any function taking two int32s and returning an int32 can be used as an element-wise operator.
`typedef float(*floatElementArithmeticFunc_t)(float a, float b);`	■ Same pattern for floats.

Index helper — calcIndicesByRawIndex():

`void calcIndicesByRawIndex(size_t numberOfDims, size_t *dims, size_t rawIndex, size_t *indices) {`	■ Convert a flat element index into a per-dimension index array.
`size_t offset = 1;`	■ Will hold the product of all dimensions (total elements).
`for (size_t i = 0; i < numberOfDims; i++) { offset *= dims[i]; }`	■ Compute total element count.
`size_t restIndex = rawIndex;`	■ Running remainder.
`for (size_t i = 0; i < numberOfDims; i++) {`	■ Peel off each dimension from most-significant to least.
`offset /= dims[i];`	■ offset now equals the stride for dimension i.
`indices[i] = restIndex / offset;`	■ How many full strides of dimension i fit in restIndex.
`restIndex -= indices[i] * offset;`	■ Remove the contribution of this dimension from the remainder.
`}`	
`}`	■ Example: dims=[2,3], rawIndex=4 -> offset starts at 6; i=0: offset=3, indices[0]=1, rest=1; i=1: offset=1, indices[1]=1. -> (1,1).

The generic float point-wise driver — floatPointWiseArithmetic():

`void floatPointWiseArithmetic(tensor_t *aTensor, tensor_t *bTensor,`	■ Takes two input tensors, a function pointer, and an output tensor.
`floatElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor) {`	■ arithmeticFunc is called once per element pair — e.g. addFloat32s, mulFloat32s.
`if (!doDimensionsMatch(aTensor, bTensor)) { exit(1); }`	■ Dimension check — both tensors must have the same shape.
`for (size_t i = 0; i < numberOfElements; i++) {`	■ Iterate over all elements by flat index.
`calcIndicesByRawIndex(..., i, aIndices);`	■ Convert flat index i to per-dimension indices for tensor A.
`aElementIndex = calcElementIndexByIndices(..., aTensor->shape->orderOfDimensions);`	■ Map logical indices to physical storage index, respecting transposition.
`// same for bTensor`	
`float aValue = readBytesAsFloat(&aTensor->data[aByteIndex]);`	■ Read element from A's raw byte buffer using the DTypes helper.
`float bValue = readBytesAsFloat(&bTensor->data[bByteIndex]);`	■ Read element from B.

`float result = arithmeticFunc(aValue, bValue);`	■ Apply the operator — e.g. aValue + bValue.
`writeFloatToByteArray(result, &outputTensor->data[outputByteIndex]);`	■ Write result back to the output buffer.
`}`	
`}`	

✔ *The InPlace variants write results back into aTensor->data instead of a separate outputTensor, saving an allocation.*

---

10 · Matmul.h / Matmul.c

`src/arithmetic/include/Matmul.h + src/arithmetic/Matmul.c`

Matmul implements 2-D matrix multiplication for float32, int32, and sym_int32 tensors. The float variant is the hottest path in the training loop — every forward and backward pass through a Linear layer calls it. The sym_int32 variant multiplies integer matrices and then propagates the scale factor.

`void matmulFloatTensors(tensor_t *aTensor, tensor_t *bTensor, tensor_t *outputTensor) {`	■ Core float matmul: output = A x B.
`// Determine aRows, aColumns, bRows, bColumns from shape`	
`if (aColumns != bRows) { exit(1); }`	■ Compatibility check — inner dimensions must match.
`for (size_t rowIndex = 0; rowIndex < aRows; rowIndex++) {`	■ Outer loop over rows of A (= rows of output).
`for (size_t columnIndex = 0; columnIndex < bColumns; columnIndex++) {`	■ Middle loop over columns of B (= columns of output).
`float result = 0;`	■ Accumulator for the dot product.
`for (size_t i = 0; i < aColumns; i++) {`	■ Inner loop — dot product over the shared dimension.
`size_t aIndices[] = {rowIndex, i};`	■ Logical index into A.
`size_t aValueIndex = calcElementIndexByIndices(..., aTensor->shape->orderOfDimensions);`	■ Physical index — handles transposition if orderOfDimensions has been swapped.
`float aValue = readBytesAsFloat(&aTensor->data[aByteIndex]);`	■ Read A[rowIndex][i].
`// same for B[i][columnIndex]`	
`result += mulFloat32s(aValue, bValue);`	■ Accumulate the product.
`}`	
`writeFloatToByteArray(result, &outputTensor->data[outputByteIndex]);`	■ Write output[rowIndex][columnIndex].
`}`	
`}`	
`}`	
`void matmulSymIntTensors(tensor_t *aTensor, tensor_t *bTensor, tensor_t *outputTensor) {`	■ SYM_INT32 matmul — reuses the integer matmul then propagates scales.
`matmulInt32Tensors(aTensor, bTensor, outputTensor);`	■ Integer multiply — same triple loop but with int32 elements.
`outputSymInt32QC->scale = aSymInt32QC->scale * bSymInt32QC->scale;`	■ Scale of the product = scale_A * scale_B. This maintains the invariant: real_output = int_output * scale_output.
`}`	

✔ *The #define MATMUL_FUNC_FLOAT macro at the top lets you swap in an instruction-counting variant by compiling with -DTRACK_INSTRUCTIONS — useful for profiling on the MCU.*

11 · RNG.h / RNG.c

`src/rng/include/RNG.h + src/rng/RNG.c`

The RNG is a lightweight XOR-shift PRNG — very fast and adequate for stochastic rounding and dataset shuffling. It maintains a single module-global state variable and is NOT thread-safe (see the header comment).

`typedef struct { uint32_t state; } rng32_t;`	■ The entire RNG state is a single 32-bit unsigned integer.
`static rng32_t rng = {.state = 1};`	■ Module-global instance. static means it is invisible outside RNG.c. Initialised to 1 (state=0 is a fixed point of XOR-shift and must be avoided).
`static uint32_t rngNext(rng32_t *rng) {`	■ Core XOR-shift generator. static = file-private.
`uint32_t x = rng->state;`	■ Copy current state.
`x ^= x << 13;`	■ Left-shift XOR — spreads bits upward.
`x ^= x >> 17;`	■ Right-shift XOR — spreads bits downward.
`x ^= x << 5;`	■ Second left-shift XOR — finalises mixing. Together these three operations form the xorshift32 algorithm (Marsaglia 2003) with full 2^32-1 period.
`rng->state = x;`	■ Update state.
`return x;`	■ Return new pseudo-random 32-bit integer.
`}`	
`float rngNextFloat(void) {`	■ Convert a 32-bit integer to a float in [0, 1).
`return (float)(rngNext(&rng) >> 8) / (float)(1 << 24);`	■ Discard 8 low-order bits (>> 8 gives 24 significant bits). Divide by 2^24 to map to [0,1). 24-bit mantissa matches float32 precision.
`}`	
`void rngShuffleIndices(size_t *indices, size_t n) {`	■ Fisher-Yates shuffle of an index array.
`for (size_t i = n - 1; i > 0; --i) {`	■ Iterate from the last element down to index 1.
`size_t j = rngBounded(&rng, i + 1);`	■ Pick a random index j in [0, i] — unbiased thanks to rngBounded's rejection sampling.
`size_t tmp = indices[i];`	
`indices[i] = indices[j];`	■ Swap indices[i] and indices[j].
`indices[j] = tmp;`	■ After the loop, every permutation is equally likely.
`}`	
`}`	

✔ *rngBounded() uses rejection sampling (do{}while x >= limit) to eliminate modulo bias — important for fair dataset shuffling when n is not a power of 2.*

12 · Layer.h / Layer.c

`src/layer/include/Layer.h + src/layer/Layer.c`

Layer.h implements C-style polymorphism. A layer_t holds a type enum and a config union; a global layerFunctions[] array maps each type to its forward, backward, and shape-calculation function pointers. The training loop calls layerFunctions[layer->type].forward(layer, input, output) without knowing the concrete layer type.

`typedef enum layerType {`	■ Enumerate all supported layer types.
`LINEAR, RELU, CONV1D, SOFTMAX, SEQUENTIAL, QUANTIZATION`	■ Each enumerator is an integer index into layerFunctions[].
`} layerType_t;`	
`typedef union layerConfig {`	■ A union holds one of several typed config pointers. Only the member matching the layer's type is valid at any time.
`linearConfig_t* linear;`	■ Active when type == LINEAR.
`reluConfig_t* relu;`	■ Active when type == RELU.
`softmaxConfig_t* softmax;`	■ Active when type == SOFTMAX.
`} layerConfig_t;`	■ Unions save memory — the pointer is the same size regardless of which config it points to.
`typedef struct layer {`	
`layerType_t type;`	■ Discriminator — determines which config pointer is valid and which layerFunctions[] entry to use.
`layerConfig_t* config;`	■ Pointer to the type-specific configuration (allocated via reserveMemory).
`} layer_t;`	
`typedef void (*forwardFn_t)(layer_t*, tensor_t*, tensor_t*);`	■ Signature: takes the layer, its input, and writes to its output.
`typedef void (*backwardFn_t)(layer_t*, tensor_t*, tensor_t*, tensor_t*);`	■ Signature: takes the layer, the forward input, the upstream loss gradient, and writes the propagated loss.
`typedef void (*calcOutputShapeFn_t)(layer_t*, shape_t*, shape_t*);`	■ Compute output shape from input shape — used for pre-allocating output tensors.
`typedef struct layerFunctions {`	■ Vtable row for one layer type.
`forwardFn_t forward;`	
`backwardFn_t backward;`	
`calcOutputShapeFn_t calcOutputShape;`	
`} layerFunctions_t;`	
`extern layerFunctions_t layerFunctions[];`	■ Global vtable — defined in Layer.c, declared extern here so any file that includes Layer.h can dispatch through it.

Layer.c — the dispatch table definition:

`layerFunctions_t layerFunctions[] = {`	■ Array initialised with designated initialisers.
`[LINEAR] = {linearForward, linearBackward, linearCalcOutputShape},`	■ Wire the LINEAR vtable row to the concrete functions in Linear.c.

`[RELU] = {reluForward, reluBackward, reluCalcOutputShape},`	
`[CONV1D] = {NULL, NULL, NULL},`	■ Conv1D is a stub — calling forward would dereference NULL. Guard code should check for this.
`[SOFTMAX] = {softmaxForward, softmaxBackward, softmaxCalcOutputShape},`	
`};`	
`void initLayer(layer_t *layer, layerType_t type, layerConfig_t *config) {`	■ Trivial constructor — sets the two fields.
`layer->type = type;`	
`layer->config = config;`	
`}`	

13 · Linear.h / Linear.c

`src/layer/include/Linear.h + src/layer/Linear.c`

Linear implements the fully-connected layer: output = W * input + bias. The forward pass calls transposeTensor (O(1)) then matmul. The backward pass computes three gradients: weight gradients (dL/dW), bias gradients (dL/db), and propagated loss (dL/dx — sent to the layer below). Both float32 and SYM_INT32 paths are implemented, plus conversion wrappers for mixed precision.

`typedef struct linearConfig {`	
`parameter_t* weights;`	■ Weight matrix W. param holds the values; grad accumulates dL/dW.
`parameter_t* bias;`	■ Bias vector b.
`quantization_t* forwardQ;`	■ Quantization type for the forward output tensor.
`quantization_t* weightGradQ;`	■ Quantization type for the weight gradient tensor.
`quantization_t* biasGradQ;`	■ Quantization type for the bias gradient tensor.
`quantization_t* propLossQ;`	■ Quantization type for the propagated loss tensor.
`} linearConfig_t;`	■ Having four separate quantizations lets you run float forward but int8 gradients, or vice versa.

Forward pass — float path:

`void linearForwardFloat(tensor_t *w, tensor_t *b, tensor_t *input, tensor_t *output) {`	
`transposeTensor(w, 0, 1);`	■ Temporarily transpose W from [out, in] to [in, out] so matmul(input[1, in], W[in, out]) works correctly.
`matmulFloat32Tensors(input, w, output);`	■ output = input x W^T = W * input (broadcast over batch). Writes to output tensor.
`transposeTensor(w, 0, 1);`	■ Restore W to [out, in] — the transpose is non-destructive (just swaps orderOfDimensions).
`addFloat32TensorsInplace(output, b);`	■ output += b — broadcast bias addition.
`}`	

Backward pass — weight gradient (float path):

`void linearCalcWeightGradsFloat32(tensor_t *forwardInput, tensor_t *loss, tensor_t *weightGrads) {`	■ dL/dW = loss^T x input.
`// allocate stack tensor intermediateWGrad`	■ Stack-allocate a temporary tensor to hold one batch's contribution.

`transposeTensor(loss, 0, 1);`	■ Transpose loss from [1, out] to [out, 1] so matmul gives [out, in] weight gradient shape.
`matmulFloat32Tensors(loss, forwardInput, &intermediateWGrad);`	■ intermediateWGrad = loss^T x input = dL/dW for this sample.
`transposeTensor(loss, 0, 1);`	■ Restore loss orientation.
`addFloat32TensorsInplace(weightGrads, &intermediateWGrad);`	■ Accumulate — gradients are summed over the batch before the optimizer step.
`}`	

Backward dispatch — linearBackward():

`void linearBackward(layer_t *linearLayer, tensor_t *forwardInput, tensor_t *loss, tensor_t *propLoss) {`	
`switch (linearConfig->weightGradQ->type) {`	■ Dispatch on the weight-gradient quantization type.
`case FLOAT32:`	
`linearCalcWeightGradsFloatWithConversion(linearConfig, forwardInput, loss);`	■ The ...WithConversion variants check if the input tensors are already float; if not, they convert to float first, compute, then convert back.
`break;`	
`case SYM_INT32:`	
`linearCalcWeightGradsSymInt32WithConversion(linearConfig, loss, forwardInput);`	■ Same pattern for int32 gradients.
`break;`	
`}`	
`// identical switches for biasGradQ and propLossQ`	■ Bias grad and propagated loss are computed independently, each with their own quantization.
`}`	

14 · Relu.h / Relu.c

`src/layer/include/Relu.h + src/layer/Relu.c`

ReLU is stateless — no trainable parameters. Forward: max(0, x). Backward: pass upstream gradient through where input > 0, zero it elsewhere.

`void reluForwardFloat(tensor_t *input, tensor_t *output) {`	
`gteFloatValue(input, 0, 0, output);`	■ gteFloatValue (from Comparison.c): element-wise max(x, 0). Each element: output[i] = (input[i] >= 0) ? input[i] : 0.
`}`	
`void reluBackwardFloat(tensor_t *forwardInput, tensor_t *loss, tensor_t *propLoss) {`	
`for (size_t i = 0; i < numberOfElements; i++) {`	■ Iterate over all elements.
`if (inputArray[i] <= 0) {`	■ If the forward input was non-positive, ReLU was zero — gradient does not flow.
`gradInArray[i] = 0;`	■ Kill gradient.
`} else {`	
`gradInArray[i] = gradOutArray[i];`	■ Forward input was positive — ReLU was linear, so gradient passes through unchanged.
`}`	
`}`	

`}`	■ This is the ReLU straight-through gradient — exact for the piecewise-linear ReLU function.

15 · Softmax.h / Softmax.c

`src/layer/include/Softmax.h + src/layer/Softmax.c`

Softmax converts raw logits into a probability distribution. The implementation uses the numerically stable log-sum-exp trick: subtracting the maximum before exponentiating prevents overflow for large logits.

`static void softmaxForwardFloat(tensor_t *input, tensor_t *output) {`	■ Float-path softmax. static = file-private.
`float *x = (float *)input->data;`	■ Cast raw uint8_t* to float* for direct element access — valid because the tensor is FLOAT32.
`float *y = (float *)output->data;`	
`// 1. find max`	
`float max = x[0];`	
`for (size_t i = 1; i < n; i++) {`	
`if (x[i] > max) max = x[i];`	■ Find the largest logit.
`}`	
`// 2. exp and sum`	
`float sum = 0.f;`	
`for (size_t i = 0; i < n; i++) {`	
`float e = expf(x[i] - max);`	■ Subtract max before exponentiation. expf(x-max) is always in (0, 1], so no overflow. All relative ratios are preserved.
`y[i] = e; sum += e;`	
`}`	
`// 3. normalize`	
`for (size_t i = 0; i < n; i++) {`	
`y[i] /= sum;`	■ Divide by the sum of exps to get probabilities that sum to 1.
`}`	
`}`	
`static void softmaxBackwardFloat(tensor_t *input, tensor_t *loss, tensor_t *propLoss) {`	■ Jacobian-vector product for softmax.
`float dot = 0.0f;`	
`for (size_t i = 0; i < n; i++)`	
`dot += s[i] * dLds[i];`	■ dot = sum(softmax_i * dL/ds_i) — the dot product of the softmax output and the upstream gradient.
`for (size_t i = 0; i < n; i++)`	
`dLdx[i] = s[i] * (dLds[i] - dot);`	■ dL/dx_i = s_i * (dL/ds_i - dot). This is the exact Jacobian-vector product of softmax. In practice, when fused with cross-entropy, it simplifies further to s - y_true.
`}`	

✔ When Softmax is followed by CrossEntropy, crossEntropySoftmaxBackward replaces this and computes the fused gradient s - y_true directly — more numerically stable and O(n) instead of the full Jacobian.

16 · QuantizationLayer.c

src/layer/QuantizationLayer.c

QuantizationLayer is the FQT insertion point. Its forward pass converts a tensor from one quantization type to another (e.g. float32 → SYM_INT32). Its backward pass is a straight-through estimator — the gradient is passed back without any transformation, as if the quantization were an identity function.

`void quantization(tensor_t *input, tensor_t *output) {`	■ Forward pass — performs the quantization conversion.
`qtype_t inputQType = input->quantization->type;`	■ Read the input's quantization type.
`qtype_t outputQType = output->quantization->type;`	■ Read the target quantization type.
`conversionFunction_t conversion = conversionMatrix[inputQType][outputQType];`	■ Look up the conversion function in the 5x5 dispatch table from TensorConversion.c.
`conversion(input, output);`	■ Call the conversion — e.g. convertFloatTensorToSymInt32Tensor.
`}`	■ The backward pass (not shown here) is a no-op: copy upstream gradient directly to propLoss — the straight-through estimator treats quantization as a linear passthrough during backprop.

ase 7 — Loss Functions

LossFunction · CrossEntropy · MSE

17 · LossFunction.h / LossFunction.c

`src/loss_functions/include/LossFunction.h + LossFunction.c`

LossFunction defines the polymorphic loss interface — same dispatch-table pattern as Layer.

`typedef float (*lossFwdFn_t)(tensor_t* modelOutput, tensor_t* label);`	■ Function-pointer type for forward pass: takes model output and ground-truth label, returns scalar loss.
`typedef void (*lossBwdFn_t)(tensor_t* modelOutput, tensor_t* label, tensor_t* result);`	■ Function-pointer type for backward pass: fills result with dL/d(modelOutput).
`typedef struct lossFunctions {`	■ Vtable row for one loss type.
`lossFwdFn_t forward;`	
`lossBwdFn_t backward;`	
`} lossFunctions_t;`	
`typedef enum lossType { MSE, CROSS_ENTROPY } lossType_t;`	■ Two supported losses.
`extern lossFunctions_t lossFunctions[];`	■ Global vtable — indexed by lossType_t.
`// LossFunction.c:`	
`lossFunctions_t lossFunctions[] = {`	■ Fill the vtable at startup.
`[MSE] = {mseLossForward, mseLossBackward},`	
`[CROSS_ENTROPY] = {crossEntropyForwardFloat, crossEntropySoftmaxBackward},`	
`};`	■ The training loop calls lossFunctions[lossType].forward(output, label) without caring which loss is active.

18 · CrossEntropy.c

`src/loss_functions/CrossEntropy.c`

Cross-entropy loss for classification. Forward: -sum(y_true * log(y_hat)). Backward: fused softmax+cross-entropy gradient = y_hat - y_true.

`float crossEntropyForwardFloat(tensor_t *softmaxOutput, tensor_t *distribution) {`	
`float *p = (float *)softmaxOutput->data;`	■ Predicted probabilities (softmax output).
`float *y = (float *)distribution->data;`	■ Ground-truth label distribution (one-hot or soft label).
`float loss = 0.f;`	
`for (size_t i = 0; i < n; i++) {`	
`float pi = fmaxf(p[i], 1e-7f);`	■ Clamp to avoid log(0). fmaxf is the float version of max — avoids NaN when softmax produces a tiny probability.
`loss += y[i] * -logf(pi);`	■ -logf(pi): log of a probability in (0,1] is negative, so this is positive. Multiplied by y[i] to only count the true class(es).
`}`	
`return loss;`	■ Scalar loss value for logging.
`}`	

`static void crossEntropySoftmaxBackwardFloat(...) {`	■ Fused backward — avoids computing the full Jacobian.
`for (size_t i = 0; i < totalInputSize; i++) {`	
`lossFloat[i] = softmaxOutputFloat[i] - distributionFloat[i];`	■ Gradient of cross-entropy w.r.t. the softmax INPUT is simply (predicted - true). This cancels the softmax Jacobian — a standard result from the chain rule applied to log-softmax.
`}`	
`}`	

19 · Sgd.h / Sgd.c

`src/optimizer/include/Sgd.h + src/optimizer/Sgd.c`

SGD with optional momentum and weight decay. Two update rules: vanilla SGD (sgdStep) and SGD-with-momentum (sgdStepM).

`typedef struct sgd {`	
`float learningRate;`	■ Step size — how far to move parameters in the gradient direction.
`float momentumFactor;`	■ Momentum coefficient (0 = no momentum, 0.9 = typical). Used only by sgdStepM.
`float weightDecay;`	■ L2 regularisation coefficient. Adds weightDecay * w to the gradient, shrinking weights toward zero each step.
`} sgd_t;`	

Vanilla SGD step (float32 path):

`static void sgdStepFloat(optimizer_t *optim) {`	■ static = file-private.
`sgd_t *sgd = (sgd_t *)optim->impl;`	■ Cast the void* optimizer implementation pointer to the concrete sgd_t.
`for (size_t stateIndex = 0; stateIndex < optim->sizeStates; stateIndex++) {`	■ Iterate over all trainable parameters.
`parameter_t *param = optim->parameter[stateIndex];`	■ Get the i-th parameter.
`float *gradArr = (float *)param->grad->data;`	■ Gradient buffer.
`float *dataArr = (float *)param->param->data;`	■ Weight buffer.
`for (size_t elementIndex = 0; elementIndex < numberOfValues; ++elementIndex) {`	■ Element-wise update.
`float grad = gradArr[elementIndex] + sgd->weightDecay * dataArr[elementIndex];`	■ Effective gradient = accumulated gradient + L2 penalty. Weight decay discourages large weights.
`dataArr[elementIndex] -= sgd->learningRate * grad;`	■ SGD update: w = w - lr * grad.
`}`	
`}`	
`}`	

SGD with momentum (float path):

`static void sgdStepMFloat(optimizer_t *optim) {`	
`// ... get sgd, param, state tensors ...`	
`for (size_t elementIndex = 0; elementIndex < numberOfValues; ++elementIndex) {`	
`float grad = gradArr[elementIndex] + sgd->weightDecay * paramArr[elementIndex];`	■ Gradient with weight decay.
`stateArr[elementIndex] = sgd->momentumFactor * stateArr[elementIndex] + grad;`	■ Velocity update: v = beta * v + grad. The velocity buffer (stateArr) accumulates an exponentially weighted average of past gradients.
`paramArr[elementIndex] -= sgd->learningRate * stateArr[elementIndex];`	■ Parameter update: w = w - lr * v. Momentum smooths oscillations and accelerates convergence in consistent gradient directions.
`}`	
`}`	

`void sgdZeroGrad(optimizer_t *optimizer) {`	■ Called after each optimizer step to reset gradient accumulators.
`for (...) {`	
`memset(param->grad->data, 0, totalNumberOfBytes);`	■ Zero out all bytes of the gradient tensor.
`if (param->grad->quantization->type == SYM_INT32) {`	
`symIntQ->scale = 1.f;`	■ Reset the SYM_INT32 scale to 1 — a fresh gradient has not been quantized yet, so its scale is undefined. Setting it to 1 prevents stale scales from corrupting the next backward pass.
`}`	
`}`	
`}`	

ase — — Syntax Quick Reference

C constructs used across the codebase

Syntax construct	Example	What it means
#define SOURCE_FILE	`#define SOURCE_FILE "FOO"`	Sets the file label used in debug macros. Must appear before #include "Common.h".
#ifndef / #define / #endif guard	`#ifndef ODT_FOO_H ...`	Header guard — prevents a header from being processed more than once per translation unit.
typedef enum	`typedef enum { A, B } myEnum_t;`	Creates an enumeration type. Values default to 0, 1, 2, ... A typedef makes the name usable without the 'enum' keyword.
typedef struct	`typedef struct foo { int x; } foo_t;`	Creates a named struct type accessible as foo_t.
typedef union	`typedef union bar { int* a; float* b; } bar_t;`	All members share the same memory — only one is valid at a time (discriminated by an external type field).
Function pointer typedef	`typedef void (*fn_t)(int a);`	fn_t is now a type for a pointer to a function taking int and returning void.
Designated initialisers	`[LINEAR] = {fnA, fnB}`	C99 syntax to initialise a specific array/struct element by name rather than position.
static (file scope)	`static void helper(void) {}`	Restricts the function/variable to the current .c file — invisible to other translation units (C's version of private).
void* (type erasure)	`void* qConfig;`	Generic pointer — can hold any pointer type. Must be cast before dereferencing. Used here so quantization_t can hold any of five config struct types.
memcpy type punning	`memcpy(&x;, bytes, 4);`	The only standard-conforming way to reinterpret a byte sequence as a typed value (avoids undefined behaviour from pointer aliasing).
do{}while(false)	`do { stmt; } while(false)`	Wraps multiple statements into a single expression safe to use after an unbraced if. Used in all macro definitions.
##__VA_ARGS__	`printf(fmt, ##__VA_ARGS__)`	GNU extension: if no variadic args are given, the ## swallows the preceding comma, preventing a syntax error.
Stack VLA	`float buf[n]; tensor_t t;`	Variable-length array on the stack — valid C99. Used extensively to avoid heap allocations in hot paths on the MCU.
extern array[]	`extern layerFunctions_t layerFunctions[];`	Declaration: the array is defined in exactly one .c file; all other files that include this header share access to the same array.

Next step — Guide 3: Phases 9–13 (DataLoader, Serialization, InferenceApi, TrainingLoopApi, MnistExperiment) with the same line-by-line treatment. Guides 1 and 2 together give you complete coverage of the foundation, memory, tensor, arithmetic, RNG, layer, loss, and optimizer subsystems.